GESTIÓN DE CONFIGURACIÓN Y MANTENIMIENTO DE SOFTWARE · UNIDAD 2

Recorrido resuelto con Flask

Los formatos F-08 a F-13 leídos en un proyecto real · Maestría en Ingeniería de Software, Universidad de La Sabana

Flask es el sistema de ejemplo del docente desde la Unidad 1. Este documento no diseña una estrategia para Flask: **lee la que ya tiene**, en su propio repositorio, para mostrar cómo se ve cada formato con evidencia real. La etiqueta de referencia es la versión 3.1.3; el historial se consultó el 31 de octubre de 2026.

> **CÓMO USAR ESTE RECORRIDO**
>
> Léanlo antes de llenar sus formatos. Su documento es distinto: ustedes **diseñan** la estrategia para su sistema. Lo que sirve de aquí es el tipo de evidencia y el nivel de detalle.

```
git clone https://github.com/pallets/flask.git
python herramientas/configuracion.py inventario flask --ref 3.1.3
python herramientas/configuracion.py traza flask --desde 3.0.3 --hasta 3.1.0 --registro
CHANGES.rst
```

Los tres CSV que producen estos comandos están en la carpeta `ejemplo_flask/` del repositorio.

F-08 · Inventario de ítems de configuración

La herramienta clasifica los 235 archivos versionados en la etiqueta 3.1.3: 88 de documentación, 61 de pruebas, 42 de ejemplos, 24 de código fuente, 7 de gobierno del proyecto, 6 de entorno y herramientas, 4 de integración continua, 2 de construcción y dependencias y 1 sin categoría. Los ítems que importan para la estrategia son estos:

ID	Ítem	Categoría	Por qué es ítem de configuración
CI-COD-01	`src/flask/` (24 archivos)	Código fuente	Es el producto
CI-PRU-01	`tests/`	Pruebas	Define qué comportamiento se verifica en cada cambio
CI-DEP-01	`pyproject.toml`	Construcción y dependencias	Versión declarada, dependencias mínimas, versión de Python requerida
CI-DEP-02	`uv.lock`	Construcción y dependencias	Fija las versiones exactas con que se prueba y se construye
CI-INT-01	`.github/workflows/tests.yaml`	Integración continua	Qué se prueba en cada PR y en qué plataformas
CI-INT-02	`.github/workflows/publish.yaml`	Integración continua	Cómo se construye y publica cada versión
CI-GOB-01	`.github/pull_request_template.md`	Gobierno	Qué se exige a un cambio

ID	Ítem	Categoría	Por qué es ítem de configuración
CI-GOB-02	`CHANGES.rst`	Gobierno	Registro de cambios por versión, con referencias a PR e issues
CI-DOC-01	`docs/`	Documentación	Se publica por versión; un cambio de API exige actualizarla

Lo que vive fuera del repositorio

Elemento	Dónde vive
Paquetes publicados	PyPI, y como borrador de la entrega en GitHub
Issues, PR y sus vínculos	GitHub
Avisos de seguridad	GitHub, como avisos privados que luego se publican (la plantilla de issues lo indica)
Quién puede fusionar y publicar	La configuración del repositorio en GitHub, del entorno `publish` y del proyecto en PyPI

F-09 · Identificación y versionado

Esquema de versiones	`X.Y.Z` : la Y sube con funcionalidad nueva (3.1.0) y la Z con correcciones (3.1.1 a 3.1.3).
Etiquetas	Sin prefijo y **anotadas**: cada una registra quién la creó y cuándo. La 3.1.0 es del 13 de noviembre de 2024 y la 3.1.3 del 18 de febrero de 2026.
Ramas permanentes	`main` para la siguiente versión y `stable` para las correcciones de la vigente. `stable` se integra a `main` con frecuencia. Durante el desarrollo de la 3.1.0 la rama de mantenimiento se llamaba `3.0.x`: hay 17 integraciones con ese nombre.
Versión declarada	`pyproject.toml` declara `3.1.3` en la etiqueta 3.1.3 y `3.2.0.dev` en `main` : el artefacto dice qué es.
Identificación del artefacto	El flujo de publicación construye el paquete una sola vez, lo guarda como artefacto del flujo y lo usa tanto para la entrega en GitHub como para PyPI. Las dos publicaciones reciben el mismo archivo.
Mensajes de commit	Frases cortas en imperativo («add SESSION_COOKIE_PARTITIONED config»). Las fusiones de PR terminan con el número entre paréntesis: «(#5637)».

F-10 · Líneas base

Línea base	Evento que la establece	Qué incluye	Cómo se marca	Cómo se reconstruye
De desarrollo	Cada integración a `main` con las pruebas en verde	Todo el repositorio	El commit de `main`	`git switch --detach <commit>` y `uv sync --locked`
De mantenimiento	Cada corrección integrada a `stable`	La versión vigente con sus correcciones	El commit de `stable`	Igual que la anterior
De entrega	La creación de una etiqueta	El código, `uv.lock` y el paquete construido	Etiqueta anotada; entrega en GitHub; versión en PyPI	El flujo fija la fecha del paquete con la del último commit (`SOURCE_DATE_EPOCH`), para que construir de nuevo la misma etiqueta dé el mismo paquete

Quién aprueba la línea base de entrega no está escrito en el repositorio. La etiqueta anotada registra quién la creó, y el entorno `publish` puede exigir aprobación en GitHub, pero esa configuración no es visible desde Git.

F-11 · Control de cambios

Clases de cambio

Clase	Cuándo aplica	Qué se exige
Menor	Erratas de documentación y cambios simples que no tocan código	PR sin issue previo (la plantilla lo permite)
Normal	Cualquier cambio de código	Issue previo, PR con pruebas que fallen sin el cambio, documentación, entrada en `CHANGES.rst` con el vínculo, y pruebas automáticas en verde
De seguridad	Vulnerabilidades	Aviso privado, no un issue público. Se corrige en `stable` y sale en una versión de corrección: la 3.1.1 y la 3.1.3 citan avisos (GHSA) en el registro de cambios

Flujo de un cambio normal

Paso	Evidencia en el repositorio
Solicitud	Plantillas de issue (`bug-report.md` , `feature-request.md`); los issues en blanco están deshabilitados
Análisis y decisión	Discusión en el issue y revisión del PR, en GitHub
Implementación y verificación	`tests.yaml` corre en cada PR: en la 3.1.3, 12 combinaciones de versión de Python, sistema operativo y versiones de dependencias, más verificación de tipos
Registro y cierre	Entrada en `CHANGES.rst` ; «fixes #n» en el PR cierra el issue

INTEGRIDAD DEL PIPELINE

En la etiqueta 3.1.3, las 20 acciones externas de los cuatro flujos de CI están fijadas por hash de commit, no por etiqueta. Una acción fijada por etiqueta puede cambiar sin que el repositorio cambie; fijada por hash, no. El pipeline es un ítem de configuración y Flask lo trata como tal.

F-12 · Matriz de trazabilidad

Tres entradas reales de la versión 3.1.0, seguidas hasta el final:

Motivo	PR	Commit	Ítems afectados	Versión	Prueba	Dónde se rompe la cadena
Issue 5636: `trusted_hosts` en el enrutamiento	5637 (fusión `7b21d43d`)	`4f7156f2`	CI-COD-01, CI-DOC-01, CI-GOB-02	3.1.0	`tests/test_request.py`	Issue → PR: solo en GitHub
PR 5623: retirar Python 3.8	5623	`99ce7ed0`	CI-COD-01, CI-DEP-01, CI-INT-01, CI-DOC-01	3.1.0	Matriz de CI	No se rompe
PR 5496: opción de respuestas OPTIONS automáticas	5496	`d718ecf6`	CI-COD-01, CI-DOC-01, CI-GOB-02	3.1.0	Ninguna nueva	El commit no cita el PR: solo se encuentra buscando el contenido (`git log -S`). Tampoco agrega prueba

Medición con la herramienta

Medida, entre 3.0.3 y 3.1.0	Resultado
Entradas del registro de la 3.1.0 con referencia a PR o issue	10 de 10
Entradas que se siguen hasta un commit solo con Git	2 (PR 5623, 5624 y 5633)
Entradas que se siguen buscando el contenido del cambio	8: siete hasta el commit y el PR; una, la del PR 5496, solo hasta el commit
Fusiones con número de PR en el mensaje	51 de 70 (las otras 19 integran ramas de mantenimiento)
Commits individuales con referencia	3 de 93

La herramienta reporta 11 entradas nuevas: las 10 de la 3.1.0 y la corrección del número de issue de una entrada antigua (de 5230 a 5336), que también es un cambio de trazabilidad.

F-13 · Roles y modelo

Rol	Dónde se ve en Flask	Dónde no se ve
Responsable de configuración	Quien mantiene `uv.lock`, los flujos y el registro de cambios: se ve en el historial	No hay un rol declarado
Autoridad de cambios	Quien fusiona los PR: se ve en el historial	Quién tiene permiso de fusionar: configuración de GitHub
Desarrolladores	Autores de los commits	—
Responsable de entregas	Quien crea la etiqueta anotada	Quién aprueba el entorno `publish` y administra PyPI

Decisiones de Flask y riesgos que atienden

Riesgo (desde la Unidad 1)	Decisión que lo atiende
Una actualización rompe a quien depende de Flask al retirar API	Registro de cambios por versión y período de obsolescencia antes de retirar
Una dependencia cambia sin que el repositorio cambie	`uv.lock` y pruebas con `--locked`; acciones de CI fijadas por hash
No poder reconstruir lo que se publicó	Etiquetas anotadas, construcción reproducible y un solo paquete para GitHub y PyPI

Lo que este recorrido enseña para su estrategia

1. **Nombren lo que vive fuera del repositorio.** En Flask, permisos, aprobaciones y vínculos entre issue y PR están en las plataformas. En su sistema pasará lo mismo, y el inventario tiene que decirlo.
2. **Escriban los roles en el repositorio.** Un rol que solo existe en la configuración de una plataforma no se puede auditar.
3. **Prueben la trazabilidad con cambios reales.** Flask tiene un buen registro de cambios y aun así 8 de 10 entradas no se siguen solo con Git.
4. **Traten el pipeline como ítem de configuración.** Fijen versiones de dependencias y de acciones de CI.

Referencias

Berczuk, S., & Appleton, B. (2002). *Software configuration management patterns: Effective teamwork, practical integration*. Addison-Wesley.

IEEE. (2012). *IEEE Standard for configuration management in systems and software engineering* (IEEE Std 828-2012). IEEE.

Pallets. (2026). *Flask* [Repositorio de código fuente]. GitHub. https://github.com/pallets/flask

---